1차 보고서

26-1 캡스톤디자인과창업프로젝트B (그로쓰)

팀 번호	37
팀 이름	Optimus Prime
트랙	연구
팀장 및 팀원	강시연 (팀장, 2376008) 김지윤 (팀원, 2176101) 신우림 (팀원, 2376141)
지도교수	반효경

1. Team Info

1.1. 과제명

VM별 AI 워크로드 에너지 사용량 예측모델 설계

1.2. 팀 정보

- 팀 번호:37
- 팀 이름: Optimus Prime
- 트랙: 연구
- 지도교수: 반효경 교수

1.3. 팀 구성원

- 강시연 (2376008): 팀장, 과제제출물 및 면담 일정 관리/Task 분배, *트랙 A: 컴퓨터 자원 이용률과 에너지 소비량 사이의 비선형성 분석* 담당
- 김지윤 (2176101): 팀원, 팀 워크스페이스 (노션) 관리 및 정리, *트랙 B: 다중 워크로드 환경에서 에너지 사용 분리* 담당
- 신우림 (2376141): 팀원, 각종 자료 디자인 및 제작, *트랙 C: 실제 VM 환경으로의 모델 확장 및 검증* 담당

2.Project-Summary

2.1 문제 정의

현재 클라우드 서비스의 과금 시스템은 실제 에너지 소비가 아닌, CPU 코어 수·메모리 용량·GPU 유형 등 자원 할당에 기반하고 있다. 그러나 대규모 AI 워크로드 환경에서는 이 구조가 다음 세 가지 문제를 야기한다.

- 동일한 자원을 할당받아도 워크로드에 따라 실제 전력 소비가 수 배 차이난다. 예를 들어 경량 서버 프로세스(NodeJS, ~79 W)와 GPU 행렬 연산(GEMM, ~278 W)은 유사한 논리적 자원을 점유하지만 에너지 소비량은 크게 다르다.
- Multi-Tenant 환경에서 동시 실행 워크로드 간 자원 비율을 재분배해도 전체 시스템 전력은 거의 변하지 않는다. 즉, 자원 할당 비율은 에너지 비용의 신뢰할 수 있는 대리 지표(proxy)가 아니다.
- GPU 지배적인 AI 워크로드는 CPU 중심 전통 모델과 에너지 소비 구조가 근본적으로 다르며, 자원 유형(CPU·GPU·메모리·스토리지)마다 에너지 행동 특성도 상이하다.

따라서 본 연구는 보다 공정한 과금 체계를 위해 에너지 사용량 기준 과금을 제안하고, 워크로드 특성을 반영한 정확한 에너지 측정 모델을 제시한다.

2.2 기존 연구와의 비교

구분	기존 연구	본 연구
접근 방식	에너지모델링(수식기반), DVFS/스케줄링 최적화; CPU 전력 예측에 집중하여 하드웨어 카운터 또는 선형 사용률 모델 활용	실측 기반 실증 분석; 플랫폼 내장 통계만 활용(RAPL, nvidia-smi, cgroup)
대상 환경	단일 워크로드 또는 벤치마크	실험 목적에 맞는 워크로드 다수 선정 후 단독/동시 실행
자원 분할 효과	분할이 전력에 미치는 영향을 직접 검증하지 않음	CPU/메모리 비율 변경이 전력에 무영향임을 실증, GPU idle+utilization 성분 분리, 스토리지 fio 캘리브레이션 기반 단위 에너지 계수 도출
전력 프로파일링 안정성	동시 실행 시 워크로드별 전력 특성 변화 미검증	Solo vs Concurrent에서 프로파일 보존 확인
배포 모드 영향	CPU/GPU 배포 차이의 에너지 영향 미고려	동일 모델의 CPU vs GPU 배포 시 에너지 범주 전환 확인

-주요 비교 대상 연구:

- Ferroni et al. (2017), Hè et al. (2024): 멀티테넌트 서버 인프라의 전력 소비 모델 → 자원 분할의 전력 영향을 직접 실험하지 않음
- Guan et al. (2024, WattScope): 비침입적 전력 분해 → 자원 할당 변화에 따른 행동은 분석하지 않음
- Kakolyris et al. (2024): LLM 추론 에너지 모델 → 단일 워크로드 유형에 한정
- FaaSMeter (Sharma & Fuerst, 2024): 서버리스 에너지 프로파일링 → 자원 할당과 시스템 전력의 관계 미검증

2.3 제안 내용

2.3.1.트랙 A: 컴퓨터 자원 이용률과 에너지 소비 사이의 비선형성

제안 1: 워크로드 특성에 따른 비선형 전력 모델 수립

기존 연구들은 전력과 부하의 관계를 단순한 직선으로 모델링해왔다. 그러나 실제 시스템에서는 부하가 증가할수록 전력이 곡선 형태로 증가하는 경우가 많다. 트랙 A에서는 CPU와 GPU 각각에 대해 선형 모델과 2차 다항 모델을 모두 구성하여 실제 데이터를 더 잘 설명하는 모델을 비교한다. 특히 GPU 중심과 CPU 중심 워크로드 사이에서 이 곡선의 형태가 어떻게 다르게 나타나는지를 분류별로 분석한다.

제안 2: GEMM 교차 실험을 통한 하드웨어 아키텍처 효과 분리

동일한 GEMM 연산을 CPU와 GPU에서 각각 실행하여 전력/부하 곡선을 오버레이한다. 부하 생성 방식을 CPU는 taskset 코어 수 제한, GPU는 batch size 조절로 통제하여 하드웨어 아키텍처 차이만을 독립 변수로 격리한다. 두 곡선의 기울기 차이와 포화점 위치를 비교함으로써 비선형성이 워크로드 알고리즘 속성이 아닌 하드웨어 마이크로아키텍처에서 기인하는지 판별한다.

2.3.2.트랙 B: 다중 워크로드 환경에서의 에너지 소비 분리

9종의 AI 및 비AI 워크로드에 대해 50개 실험 구성(단독 10개 + 동시 실행 10조합 × 4비율 = 40개)을 수행하여 다음을 제안한다.

제안 1: 자원 할당 둔감성 (Allocation Insensitivity)

워크로드 조합이 고정된 상태에서 CPU 코어와 메모리 비율을 변경해도 전체 시스템 전력은 ±1.4% 이내로 변하지 않음 → 10개 조합 중 9개에서 확인; 예외 1개(Combination H)는 CPU 자원 부족으로 인한 GPU 파이프라인 병목

제안 2: 워크로드 전력 프로파일 보존 (Profile Preservation)

단독 실행과 동시 실행에서 각 워크로드의 CPU 이용률은 ±3%p, GPU 전력은 ±5W 이내 유지 → 전력 특성은 워크로드 고유의 성질이며, 동반 워크로드에 의해 변하지 않음

제안 3: 배포 모드 기반 에너지 범주 전환 (Deployment-Dependent Energy Shift)

동일한 ResNet-18 모델이 CPU 배포 시 108W(CPU-dominant), GPU 배포 시 216W(GPU-dominant)로 다름 → 에너지 범주는 애플리케이션 이름이 아니라 실행 하드웨어(배포 모드)에 의해 결정됨

제안 4: 이용률 기반 에너지 카테고리 프레임워크

GPU-dominant: SM 이용률이 주요 전력 결정 인자 (GEMM 95% → 224W vs LLM-GPU 25% → 57W), CPU-dominant: CPU 이용률이 주요 전력 결정 인자 (ResNet-CPU 46% → 54W vs NodeJS 5% → 27W), 메모리 관련 파라미터(RSS, 프레임버퍼)는 전력 크기에 미미한 영향을 미친다.

2.3.3.트랙 C: 실제 VM 환경으로의 모델 확장 및 검증

제안 1: 시스템 수준 에너지 분해 및 추론 기반 귀속 모델

측정 가능한 유일한 ground truth인 시스템 전체 에너지(E_sys)를 자원 유형별 (CPU·GPU·메모리·스토리지·기타)로 분해하고, 각 자원의 물리적 에너지 행동 특성에 근거한 귀속 규칙을 통해 워크로드별 에너지를 간접 추론한다. 이로써 직접 측정 없이도 워크로드 수준의 에너지 비용을 합리적으로 산출한다.

제안 2: 자원 유형별 독립적 귀속 전략

자원	에너지 특성	귀속 전략
CPU	사용률 비례(utilization-driven), 직접 측정 가능	RAPL 측정 active 에너지를 cgroup CPU 사용률 비율로 배분
GPU	할당 기반 idle 성분 + 활동 기반 active 성분 혼재	idle → vGPU 할당 비율, active → SM 사용률 비율로 이중 분리
메모리(DRAM)	정적 전력 지배(allocation-driven), 워크로드별 측정 수단 부재	할당 용량 비례(0.2 W/GB, LPDDR4 사양 기반)
스토리지(NVMe)	I/O 활동 비례(activity-driven)	fio 캘리브레이션 계수(β_r, β_w) × 워크로드별 I/O 볼륨

제안 3: 에너지 보존 일관성 보장 (3단계 파이프라인)

- Step 1. 시스템 에너지 분해 및 베이스라인 분리: E_sys = E_cpu + E_gpu + E_mem + E_sto + E_other; 워크로드 실행 전 idle 상태에서 자원별 베이스라인 에너지를 측정하고 고정 기준값으로 설정
- Step 2. 자원별 워크로드 에너지 귀속: 각 자원의 에너지 행동 특성에 맞는 독립적 귀속 전략을 적용하여 워크로드별 에너지 산출
- Step 3. 에너지 보존 일관성 보장: E_sys = E_baseline + Σ E_wi (i = 1, ..., n); 모든 귀속 에너지의 합과 베이스라인의 합이 시스템 실측 에너지와 일치함을 수식으로 보장

2.4 기대 효과 및 의의

- 에너지 인식 과금 모델의 실증적 근거 제공: 할당 기반 과금이 에너지 비용과 괴리됨을 실험으로 입증하여, 워크로드 이용률 기반 과금으로의 패러다임 전환 근거 마련
- 클라우드 운영 최적화: 워크로드 전력 프로파일이 동시 실행에서도 보존되므로, 단독 측정 결과만으로 Multi-Tenant 환경의 전력을 예측 가능, 즉, 명확한 에너지 소비 모델링을 통해 다수의 이용자가 컴퓨팅 자원을 공유하는 상황으로 확장 가능
- 학문적으로 에너지 인식 스케줄링 및 자원 관리 연구의 실험적 기반 제공

2.5 주요 기능 리스트

2.5.1.트랙 A: 컴퓨터 자원 이용률과 에너지 소비 사이의 비선형성

기능 1. 실험 전 환경 자동 검증: 실험 재현성을 보장하기 위해 실험 시작 전 시스템 환경을 자동으로 점검한다.

기능 2. 실시간 메트릭 수집: 100ms 샘플링 간격으로 9종 메트릭을 백그라운드 스레드에서 병렬 수집하여 CSV로 저장한다.

기능 3. 실험 오케스트레이션: 단일 실험 사이클의 전 과정을 자동으로 실행하며, 6종 워크로드 × 5회 반복을 순차 처리한다.

기능 4. 데이터 분석 및 시각화: 수집된 CSV를 로드하여 6종 그래프와 4단계 모델링 파이프라인을 실행한다.

2.5.2.트랙 B: 다중 워크로드 환경에서의 에너지 소비 분리

기능 1. 9종 워크로드 스위트: GPU-dominant 5종(GEMM, Training, ResNet-GPU, YOLO, LLM-GPU) + CPU-dominant 4종(ResNet-CPU, LLM, ffmpeg, NodeJS

기능 2. 자원 분할 실험 프레임워크: cgroup v2로 VM-A/VM-B 에뮬레이션; 1:1·2:1·3:1·5:1 비율로 동일 총 용량(20코어·28GB) 유지하며 분할

기능 3. 전력 분해 분석: 컴포넌트별 전력 분해 및 워크로드 조합별 Wall Power 변동 분석(10개 조합 × 4비율 = 40 실험)

기능 4. 에너지 카테고리 분류: GPU-dominant / CPU-dominant 분류 프레임워크; 배포 모드에 따른 에너지 범주 전환 확인

기능 5. 프로파일 보존 검증: 단독 vs 동시 실행에서 CPU 이용률 ±3%p, GPU 전력 ±5W 이내 유지 여부 정량 검증

기능 6. 시각화 및 보고: matplotlib 기반 14종 분석 그래프 자동 생성; summary.csv(50행) 및 7종 분석용 CSV 내보내기

2.5.3.트랙 C: 실제 VM 환경으로의 모델 확장 및 검증

기능1. 시스템 에너지 분해 및 베이스라인 분리: E_sys = E_cpu + E_gpu + E_mem + E_sto + E_other; idle 베이스라인 측정(RAPL·nvidia-smi·BIOS DIMM 비활성화) 후 고정 기준값으로 설정

기능2. CPU 에너지 귀속: E_wi^cpu = E_W^cpu × (U_i^cpu / Σ U_j^cpu); RAPL active 에너지를 cgroup v2 CPU 사용률 비율로 배분

기능3.GPU 에너지 귀속(이중 구조): E_wi^gpu = E_idle^gpu × a_i + E_W^gpu_act × (U_i^gpu / Σ U_j^gpu); 할당 기반 idle 성분(vGPU 비율)과 활동 기반 active 성분(SM 사용률) 분리

기능4. 메모리 에너지 귀속: E_wi^mem = E_idle^mem × (m_i / M); LPDDR4 사양(0.2 W/GB) 기반 할당 용량 비례 모델

기능5. 스토리지 에너지 귀속: E_wi^sto = β_r × read_i + β_w × write_i; fio 캘리브레이션으로 β_r·β_w 산출(NVMe: 2.5 J/GB); cgroup v2 I/O 통계로 워크로드별 I/O 볼륨 수집

기능6. 에너지 보존 검증 및 과금 비교: E_sys = E_baseline + Σ E_wi 조건을 수치로 확인; 동일 자원 할당 조건에서 할당 기반 과금 vs 에너지 기반 과금 차이를 배수(×) 단위로 정량화

3. Project-Design

3.1 요구사항 정의

3.1.1.트랙 A: 컴퓨터 자원 이용률과 에너지 소비 사이의 비선형성

ID	요구사항	우선순위	설명
R1	실험 전 환경 자동 검증	필수	● 워크로드 스크립트 존재 여부 및 Python 의존성(라이브러리 import, CUDA 런타임) 확인 ● CPU/GPU 통제 조건 검증: Intel Turbo Boost·C-State disable 여부, GPU/메모리 클럭 고정, CUDA_VISIBLE_DEVICES 설정 ● 계측 도구 가용성 확인: RAPL, sensors 명령 응답 확인; 결과를 항목별로 PASS/FAIL/WARN으로 분류 출력
R2	실시간 매트릭 수집	필수	● CPU 전력: RAPL(/sys/class/powercap/intel-rapl) 에너지 카운터 차분으로 순간 전력 산출 ● GPU 전력: nvidia-smi --query-gpu=power.draw ● CPU Busy / GPU Utilization: /proc/stat 및 nvidia-smi ● Memory Bandwidth Utilization, IPC: perf stat
R3	실험 오케스트레이션	필수	● 온도 기반 Idle 진입 판정: CPU ≤ 45°C, GPU ≤ 50°C 10초 이상 유지 ● CPU 중심 워크로드: taskset 코어 수 제한 + numactl 단일 노드 바인딩으로 부하 제어 ● GPU 중심 워크로드: batch size 조절로 GPU SM 점유율 제어 ● 실험 사이클: Warm-up(측정 제외) → 부하율별 워크로드 실행 → Cool-down
R4	데이터 분석 및 시각화	필수	● 그래프 1: CPU Busy – CPU 전력 회귀 분석 ● 그래프 2: GPU Utilization – GPU 전력 회귀 분석 ● 그래프 3: 워크로드별 전력 구성 비율 ● 그래프 4: Memory Boundness와 에너지 소비 간 상관관계 ● 그래프 5: 부하율별 에너지 추이 ● 그래프 6: 교차 워크로드(GEMM) 비교

3.1.2.트랙 B: 다중 워크로드 환경에서의 에너지 소비 분리

ID	요구사항	우선순위	설명
R1	9종 워크로드 자동 실행	필수	GPU 워크로드 5종 + CPU 워크로드 4종의 단독/동시 실행 스크립트
R2	4가지 자원 비율 테스트	필수	1:1, 2:1, 3:1, 5:1 비율로 CPU 코어/메모리를 분할하여 실험
R3	컴포넌트별 전력 분해	필수	idle 전력을 baseline으로 active 전력(ΔCPU, ΔGPU, ΔDRAM) 분리

ID	요구사항	우선순위	설명
R4	에너지 카테고리 분류	필수	GPU-dominant / CPU-dominant 분류 프레임워크
R5	다중 계층 동시 전력 측정	필수	CPU(RAPL), GPU(nvidia-smi), DRAM(RAPL), Wall(외부 전력계)을 1초 간격으로 동시 측정
R6	그래프 생성	필수	matplotlib 기반 14종 분석 그래프 자동 생성

3.1.3.트랙 C: 실제 VM 환경으로의 모델 확장 및 검증

ID	요구사항	우선순위	설명
R1	워크로드 수준 에너지 귀속 가능성	필수	공유 자원 환경에서 물리적 분리 측정이 불가능한 조건에서도 워크로드별 에너지 소비를 합리적으로 귀속할 수 있는 추론 기반 귀속 방법론 제시
R2	자원 유형별 이질적 에너지 행동 반영	필수	균일 규칙이 아닌 자원 유형별 구분된 귀속 전략으로 CPU·GPU·메모리·스토리지의 상이한 에너지 특성 반영
R3	AI 워크로드 에너지 특성 규명	필수	현대 AI 추론 워크로드(YOLO, ResNet, GPT2)의 에너지 소비 구조가 전통 워크로드(Node.js)와 어떻게 다른지 자원 구성 수준에서 실증적으로 규명
R4	멀티테넌트 환경에서의 귀속 유효성	필수	두 워크로드 동시 실행 공유 환경에서 제안 모델의 귀속 결과가 격리 실행 실측값과 얼마나 일치하는지 정량적으로 검증
R5	추가 계측 오버헤드 금지	필수	별도 하드웨어 instrumentation 없이 Intel RAPL, nvidia-smi/NVML, Linux cgroup v2 등 플랫폼 내장 통계만으로 동작
R6	에너지 보존 원칙 준수	필수	모든 귀속 에너지의 합과 베이스라인의 합이 시스템 실측 에너지와 수식 수준에서 일치
R7	자원 유형별 독립적 귀속 전략	필수	CPU·GPU·메모리·스토리지에 각 자원 특성에 맞는 귀속 방식 분리 적용; GPU는 반드시 할당+활동 이중 구조로 모델링
R8	격리 실행 기반 기준값 확보	필수	각 워크로드를 격리 실행하여 idle 성분을 제거한 workload-induced 에너지를 실측(동일 조건 5회 반복 평균)하고 ground truth로 사용
R9	6가지 시나리오 포괄적 검증	필수	AI+전통(3종) 및 AI+AI 대칭·비대칭(3종) 조합을 포함한 총 6가지 동시 실행 시나리오에서 귀속 정확도 검증
R10	과금 불형평성 정량화	필수	동일 자원 할당 조건에서 워크로드 간 에너지 차이를 배수(×) 단위로 측정하여 할당 기반 과금의 한계를 실증

3.2 전체 시스템 구성

3.2.1.트랙 A & 트랙 B

물리 장치 구성

장치	사양
실험 서버	Intel Xeon E5-2630 v4 (10코어, HT, 총 20 논리코어) / NVIDIA Titan V × 2 / DDR4 2133MHz 32GiB / Samsung 860 1TB SSD + HGST 4TB HDD
전력 측정 서버 (Raspberry Pi)	외부 전력계와 USB 시리얼 연결; Wall-level 전력(W) 1초 간격 기록
개발/분석 PC	SSH로 실험 서버 접속하여 실험 제어; 데이터 다운로드 후 분석·시각화

소프트웨어 계층 구성

계층	구성 요소	기술
OS/인프라	서버 OS	Ubuntu (Linux cgroup v2 지원)
자원 격리	VM 에뮬레이션	cgroup v2 (cpuset + memory.max)
GPU 관리	전력/이용률 측정	nvidia-smi, CUDA 드라이버
CPU 관리	주파수/전력 제어	RAPL, ondemand governor, Turbo Boost OFF
워크로드 (트랙 A)	AI 프레임워크	PyTorch (GEMM, Training, ResNet, LLM, GPU-LLM)
	추론 엔진	YOLOv8-nano, OPT-1.3B (HuggingFace)
	비 AI	ffmpeg (x264), Node.js (Express)
워크로드 (트랙 B)	CPU 중심	Matrix Multiplication, GROMACS
	GPU 중심	ResNet, gpt2
	교차	GEMM
데이터 처리	분석 스크립트	Python (pandas, matplotlib, numpy)

3.2.3.트랙 C: 실제 VM 환경으로의 모델 확장 및 검증

공통 실험 환경

구성 요소	세부 사양
서버 플랫폼	Alienware Aurora R12
CPU	Intel Core i7-11700KF (8코어/16스레드, TDP 125W)

GPU	NVIDIA GeForce RTX 3060 × 2 (TDP 170W each)
메모리	32GB DDR4-3467
스토리지	Samsung SSD 980 500GB NVMe
시스템 에너지 측정	RPICT4V3 + CT 센서 + Raspberry Pi (AC 입력)
CPU 에너지 측정	Intel RAPL on-chip energy counter
GPU 에너지 측정	NVIDIA nvidia-smi / NVML 전력 시계열 적분
자원 격리	Linux cgroup v2 (워크로드별 2 CPU 코어, 4GB 메모리, 전용 GPU)
평가 워크로드	YOLO(객체 탐지), ResNet(이미지 분류), GPT2(언어 생성), Node.js(웹 서비스)
실험 반복	동일 조건 5회 반복, 평균값 보고

전체 실험 시나리오

시나리오	실행 조합	검증 **Feature**	귀속 오차 결과
격리 실행(기준값)	YOLO/ResNet/GPT2/Node.js 각 단독	기능1~기능5 기준값 확보	(ground truth)
AI + 전통(이종)	YOLO+Node.js, ResNet+Node.js, GPT2+Node.js	기능2, 기능3, 기능6	AI: 0.1~0.4%, Node.js: 0.4~4.1%
AI + AI(비대칭)	YOLO+ResNet, ResNet+GPT2	기능3, 기능6	2.4~5.2%(평균 3.7%)
AI + AI(대칭)	YOLO+GPT2	기능3, 기능6	1.8~2.0%

전체 최대 귀속 오차는 약 5% 이내이며, 할당 기반 과금 대비 에너지 기반 과금의 차이는 AI+Node.js 조합 기준 4.7~11.0배로 측정되었다. 이는 제안 모델이 실용적으로 허용 가능한 오차 범위 내에서 에너지 기반 공정 과금의 실현 가능성을 실증함을 의미한다.